Model Kill-Switch Protocol | Architecture

The system is split into components, each with one job and clear interfaces. Control decides, distribution spreads, enforcement acts, observation watches, and a cross-cutting layer holds keys. No enforcement path leaves the system boundary.

3.1 Overview: components, roles, interfaces

Component	Plane	Role	Boundary
Authorized signer client	Control plane	The tool where committee members start a command. Collects partial signatures from several people and merges them into one.	Inside
Control gateway	Control plane	The only door commands enter through. Checks signature, one-time number and time, and consults the policy engine.	Inside
Policy engine	Control plane	Decides, by written rules, who may do which action on which scope.	Inside
Transparency log	Control plane	Every approved command is recorded here and cannot be erased or altered. Anyone outside can verify it.	Inside
Distribution bus	Distribution	A pipe that carries every state change to every enforcement gateway, in every region, within seconds and in the same order.	Inside
Enforcement gateway fleet	Enforcement plane	A guard next to every server that serves the model. Decides on each request: pass or block.	Inside
Local state cache	Enforcement plane	A copy of the last known state, kept on each gateway's own disk.	Inside
Model serving layer	Enforcement plane	The existing infrastructure that runs the model and answers users. Not part of the system we build.	External
Self monitoring	Observation	Watches each gateway's serving metrics and signing patterns and computes how unusual something is.	Inside

Component	Plane	Role	Boundary
Security event log	Observation	Collects every event from every component: success, rejection, anomaly. The team and other systems read it.	Inside
Key management	Cross-cutting layer	Holds the signing keys and the certificates the components use to identify each other. Rotates them automatically.	Inside

Links

- Authorized signer client → Control gateway
- Control gateway → Policy engine
- Control gateway → Transparency log
- Control gateway → Distribution bus
- Distribution bus → Enforcement gateway fleet
- Enforcement gateway fleet → Model serving layer
- Enforcement gateway fleet → Local state cache
- Enforcement gateway fleet → Self monitoring
- Self monitoring → Control gateway
- Key management → Control gateway
- Control gateway → Security event log
- Enforcement gateway fleet → Security event log

3.2 Mapping to the spec

#	Feature	Components
1	Signing every command	Authorized signer client, Control gateway, Key management
2	Never replaying a command	Control gateway
3	The gateway	Enforcement gateway fleet, Model serving layer
4	A committee, not one person	Authorized signer client, Control gateway, Key management
5	Targeted stop	Policy engine, Control gateway, Enforcement gateway fleet
6	When infrastructure fails	Enforcement gateway fleet, Local state cache
7	Guarding the guard	Self monitoring, Security event log, Control gateway
8	How many regions	Distribution bus, Enforcement gateway fleet
9	Developer libraries	Enforcement gateway fleet, Model serving layer

#	Feature	Components
10	Simulation before stopping	Authorized signer client, Policy engine, Control gateway
11	Automatic expiry	Control gateway, Transparency log, Distribution bus, Enforcement gateway fleet

3.3 Data design

Transient

- **SignedKillCommand:** Command on its way from the tool to the gateway. Not stored as-is.
- **PolicyDecision:** The question and answer between gateway and policy engine.
- **StatePropagationEvent:** The small message published on the bus to every gateway.
- **Heartbeat:** Liveness pulse. Only the time of the last one is kept.

Persisted

- **LedgerEntry:** Every approved command, written to the transparency log, never deletable. The official record of what happened and when.
- **GatewayLocalState:** Current state, stored both centrally and at each gateway. Deliberate duplication for when the network fails.
- **AnomalyEvent:** Every anomaly monitoring detected, with its score, to retune what normal means.
- **RejectionRecord:** Every rejection is kept in the security log, sometimes in the public log too. Visible rejections build trust.

Architecture decisions

- **Send directly to every gateway, or use a distribution bus?** Chosen: A distribution bus
- **One central gateway, or a guard next to every server?** Chosen: A guard next to every server
- **Ask the center on every request, or keep a local copy?** Chosen: A local copy
- **Should internal monitoring stop the model by itself?** Chosen: Monitoring only requests; it never stops

Build order

- Sprint 1: a working skeleton (one signed command stops one model, end to end)
- Sprint 2: full verification (3-of-5 committee, one-time number, policy engine)
- Sprint 3: transparency and distribution (public log, bus, multiple regions)
- Sprint 4: resilience (local cache, dead-man switch, circuit breaker)
- Sprint 5: observation (self-monitoring and security log)

Closure test

Closed end to end. The only element outside the boundary is the model serving layer, which is existing infrastructure. Everything that decides, spreads and enforces is inside. Even the self-monitor can only file a request through the same gateway and signature chain as everyone else.